Search models, datasets, users...

Join the Hugging Face community

and get access to the augmented documentation experience

Sign Up to get started

Fine-tuning

Copy page

Fine-tuning continues training a large pretrained model on a smaller dataset specific to a task or domain. For example, fine-tuning on a dataset of coding examples helps the model get better at coding. Fine-tuning is identical to pretraining except you don't start with random weights. It also requires far less compute, data, and time.

The tutorial below walks through fine-tuning a large language model with Trainer.

Log in to your Hugging Face account with your user token to push your fine-tuned model to the Hub.

```
from huggingface_hub import login

login()
```

Tokenization

Load a dataset and tokenize the text column the model trains on (`horoscope` in the dataset below).

Ask HuggingChat

karthiksagarn / **astro_horoscope**

Split (1)
train · 22k rows

Search this dataset

sign string · *classes* 12 values	category string · *classes* 5 values	date string · *classes* 366 values	horoscope string · *lengths* 189 952
aries	general	2020/06/17	There's a great day ahead of you, Aries. You'll be…
aries	general	2020/06/18	People will understand and appreciate your desire to…
aries	general	2020/06/19	You are very interested in technological…
aries	general	2020/06/20	Stress from overwork could have you feeling weaker…
aries	general	2020/06/21	This is a good day to stand up for yourself and…
aries	general	2020/06/22	The planets align to make this a great day for you,…

< Previous 1 2 3 ... 220 Next >

The tokenizer creates the model inputs, `input_ids` and `attention_mask`. The model's forward method only accepts `input_ids` and `attention_mask`, so set `remove_columns` to drop columns like `horoscope` after tokenization.

- Set `truncation=True` and a `max_length` to truncate longer sequences to a specified maximum length.
- Use the `train_test_split` method to create a test split for evaluating the model.

```
from datasets import load_dataset
from transformers import AutoTokenizer, DataCollatorForLanguageModeling

model_name = "Qwen/Qwen3-0.6B"
tokenizer = AutoTokenizer.from_pretrained(model_name)
dataset = load_dataset("karthiksagarn/astro_horoscope", split="train")

def tokenize(batch):
    return tokenizer(
```

```
        batch["horoscope"],
        truncation=True,
        max_length=512,
    )

dataset = dataset.map(tokenize, batched=True, remove_columns=dataset.column_names)
dataset = dataset.train_test_split(test_size=0.1)
```

A data collator assembles dataset samples into batches for the model to process. DataCollatorForLanguageModeling *dynamically* pads each batch to the longest sequence in that batch rather than padding every sequence in the dataset to the same length. This saves compute and memory by avoiding computing unnecessary padding tokens.

- Set `mlm=False` to avoid randomly masking tokens.

```
data_collator = DataCollatorForLanguageModeling(tokenizer, mlm=False),
```

Loading a model

Load a pretrained checkpoint to fine-tune (see the Loading models guide for more details about loading models).

- Set `dtype="auto"` to load the weights in their saved dtype. Without it, PyTorch loads weights in `torch.float32`, which doubles memory usage if the weights are originally `torch.bfloat16`.

```
from transformers import AutoModelForCausalLM, TrainingArguments, Trainer

model_name = "Qwen/Qwen3-0.6B"
model = AutoModelForCausalLM.from_pretrained(model_name, dtype="auto")
```

Training configuration

TrainingArguments provides all the options for customizing a training run. Only the most common arguments are covered here. Everything else has reasonable defaults or is only relevant to specific scenarios like distributed training. See the TrainingArguments API docs for a complete list of arguments.

training duration | training optimizations | evaluation and checkpointing | logging

- `num_train_epochs` and `per_device_train_batch_size` control training duration and batch size. `learning_rate` sets the initial learning rate for the optimizer.

```
TrainingArguments(
    output_dir="qwen3-finetuned",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    gradient_accumulation_steps=8,
    gradient_checkpointing=True,
    bf16=True,
    learning_rate=2e-5,
    logging_steps=10,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
)
```

Training

Create a Trainer instance with all the necessary components, then call train() to begin.

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset["train"],
    eval_dataset=dataset["test"],
    processing_class=tokenizer,
    data_collator=DataCollatorForLanguageModeling(tokenizer, mlm=False),
)

trainer.train()
trainer.push_to_hub()
```

push_to_hub() uploads the fine-tuned weights, generation config, tokenizer, and model config to the Hub.

Next steps

- Read the Subclassing Trainer methods guide to learn how to subclass Trainer methods to support new and custom functionalities.
- Read the Callbacks guide to learn how to hook into training events for logging, early stopping, and other custom behavior.
- Read the Data collators guide to learn how to customize how samples are assembled into batches.
- Browse transformers/examples/pytorch, notebooks, or the **Resources > Task Recipes** section for additional training examples on different text, audio, vision, and multimodal tasks.

</> Update on GitHub

← Trainer overview

Subclassing Trainer methods →